

Table des matières

1. Pour commencer
2. Premier pas avec Lex
3. Polonaise inverse et pile
4. Variables et tables des symboles

1. Pour commencer

Dans les TP, nous allons utiliser les outils Lex&Yacc. Avant toute chose, il est donc important de les installer sur votre machine.

Il existe plusieurs versions : Lex ou Flex et Yacc ou Bison. Flex et Bison sont les versions GNU.

Si vous utilisez Ubuntu ou la WSL, tapez simplement les commandes :

```
sudo apt-get install flex  
sudo apt-get install byacc
```

Vérifiez en tapant les commandes (avec le paramètre "v", par exemple) que l'installation est correcte.

2. Premier pas avec Lex

Pour créer un analyseur lexical avec *Lex*, il est conseillé de créer un fichier dont l'extension est **lex**. Ce fichier est compilé à l'aide de la commande **flex**. Cela génère un fichier C nommé **lex.yy.c**. Pour démarrer l'analyseur, il faut appeler la méthode **yyllex**.

1. Téléchargez l'archive d'exemple ([example.zip](#)) et compilez les sources. Exécutez le programme généré.

A partir de cet exemple, nous allons ajouter plusieurs fonctionnalités. Pour le moment, l'analyseur se contente d'ignorer toutes les saisies grâce à l'unique règle :

```
. ;
```

Le lexème (les caractères du texte d'entrée) correspondant au modèle (une expression régulière étendue) est stocké dans la variable **yytext**. Ici, le "." correspond à tout caractère et le ";" indique simplement que l'on ne réalise aucune action.

2. Ajoutez une règle pour reconnaître des nombres entiers et les afficher à l'écran.
3. Faites de même pour afficher les quatre opérateurs de base : "+", "-", "*", et "/".

Vous êtes maintenant prêts à utiliser *Lex* pour votre première application : une calculatrice qui utilise la notation polonaise inverse.



Une polonaise inversée (Marie Curie)

3. Polonaise inverse et pile

La notation polonaise inverse (ou notation post-fixée) permet d'évaluer des expressions sans utiliser de parenthèses. Le principe repose sur une pile dans laquelle les opérandes sont stockés. Lorsqu'on effectue une opération, on récupère les deux valeurs au sommet de la pile que l'on remplace par le résultat.

Voici quelques exemples :

- Pour l'opération "3+5", on saisit "3" "5" et "+"
 - "3" : la valeur 3 est empilée
 - "5" : la valeur 5 est empilée
 - "+" : on dépile 5 et 3, on effectue l'opération 3+5 puis on empile le résultat (8)
- L'opération (2 + 3) * (4 + 5) : on saisit "2", "3", "+", "4", "5", "+", "*"

Pour résumer, lorsqu'un entier est saisi, il est empilé. Lorsqu'un opérateur est saisi, on dépile 2 entiers puis on empile le résultat de l'opération.

1. Créez une structure pile d'entiers (`stack_t`) avec les opérations classiques : `isempty`, `isfull`, `pop`, `push`.

Pour créer rapidement une pile d'entiers, créez une structure contenant un tableau d'entiers et un sommet. Le sommet indique l'indice du tableau contenant la prochaine valeur. La pile est vide si le sommet est à 0. La pile est pleine si le sommet est égal à la taille du tableau.

2. Quand et où la pile doit-elle être créée pour être utilisée dans le *Lex* ?
3. Modifiez votre fichier *Lex* : les entiers sont empilés (sauf si la pile est pleine...) et un opérateur entraîne une opération (en vérifiant que le nombre d'entiers est suffisant). On affiche l'opération et son résultat.
4. Si ce n'est pas le cas, gérez les entiers négatifs.
5. Sans le faire, que devrait-on modifier pour pouvoir gérer les nombres réels ?
6. Modifiez votre programme pour que le retour à la ligne ne provoque pas d'erreur.
7. Faites en sorte que le caractère "=" affiche le contenu de la pile.
8. Prenez en compte les mots-clés SWAP et CLEAR qui permettent respectivement d'inverser les deux entiers au sommet de la pile et de vider la pile.

4. Variables et tables des symboles

Nous souhaitons gérer les variables. Pour cela, il faut que la pile stocke maintenant des entiers et des variables. Nous avons également besoin d'une table des symboles pour stocker les variables. Cette dernière doit être réutilisée pour les TP suivants ET [le projet](#). Pour cela, il faut trouver une structure générique pour un symbole : il est caractérisé au minimum par un nom, un type et une valeur (cette dernière dépend du type).

1. Développez une table des symboles.
2. Modifiez votre pile pour accepter les noms de variables.
3. Modifiez votre fichier lex pour supporter les variables. La commande STORE permet de stocker une variable dans la table des symboles (dans la pile, on a le nom puis la valeur).